PORTFOLIO RISK · PYTHON PROJECT · SELF-STUDY GUIDE

Value at Risk from Scratch

How to measure, test and explain the risk of an investment portfolio, built step by step in Python on a real Rs 1 crore Indian portfolio.

Rs 1.90 lakh
1-day 99% VaR

43.8%
risk removed by diversification

5.7x
worst day vs VaR (23 Mar 2020)

Reliance · HDFC Bank · ICICI Bank · Infosys · TCS · ITC · L&T · Bharti Airtel · Gold ETF · US Dollar

8 Python files | 6 VaR models | backtesting, stress tests and a live dashboard | November 2026

Contents

PART 1 — THE IDEAS

1. Returns and volatility
2. Correlation and diversification
3. Fat tails: why the bell curve lies
4. Value at Risk (VaR)
5. Expected Shortfall (ES)
6. Backtesting a risk model
7. Risk decomposition
8. Stress testing
9. How to analyse any portfolio: a checklist

PART 2 — BUILDING THE PROJECT

10. Setup: folders, VS Code and libraries
11. Step 1a · config.py — the settings
12. Step 1b · data_loader.py — prices and returns
13. Step 2 · var_basic.py — historical and parametric VaR
14. Step 3 · var_advanced.py — Expected Shortfall and Monte Carlo
15. Step 4 · backtest.py — does the model work?
16. Step 5 · risk_decomposition.py — who drives the risk?
17. Step 6 · stress_test.py — crises and what-ifs
18. Step 7 · app.py — the interactive dashboard

PART 3 — RESULTS AND REFERENCE

19. The ten findings of this project
20. Errors we hit and how we fixed them
21. Command and formula cheat sheet
22. Glossary
23. Where to go next

PART 1

The ideas

Everything you need to understand before (and after) writing a single line of code. Each idea here becomes one step of the project in Part 2.

1. Returns and volatility

Risk analysis never works on prices. It works on **returns**, the % change from one day to the next, because returns can be compared across assets with very different prices.

Daily return

If HDFC Bank closes at Rs 1,600 today and Rs 1,632 tomorrow, the simple return is 1,632 / 1,600 − 1 = **2.0%**. This project uses the **log return** instead:

$$\text{log return} = \ln(P_{today} / P_{yesterday}) = \ln(1.02) = 1.98\%$$

For small moves the two are almost identical. Log returns are preferred because they **add up over time**: a 10-day log return is simply the sum of ten daily log returns. That makes multi-day risk easy to calculate.

Volatility

Volatility is the standard deviation of returns: how much the asset typically moves in a day. A stock with 1.5% daily volatility will often move 1–2% in a day; a stock with 0.5% rarely moves more than 1%.

$$\text{annual volatility} = \text{daily volatility} \times \sqrt{252}$$

There are about **252 trading days** in a year (250 is often used for simplicity). Your portfolio's daily volatility was about 0.79%, which is about **12.6% a year**.

> **Key idea: the square-root-of-time rule**
>
> Risk does not grow in a straight line with time, because good days and bad days partly cancel out. If returns are independent from day to day, risk grows with the **square root** of the number of days. A 10-day risk is therefore about √10 = **3.16 times** the 1-day risk, not 10 times.

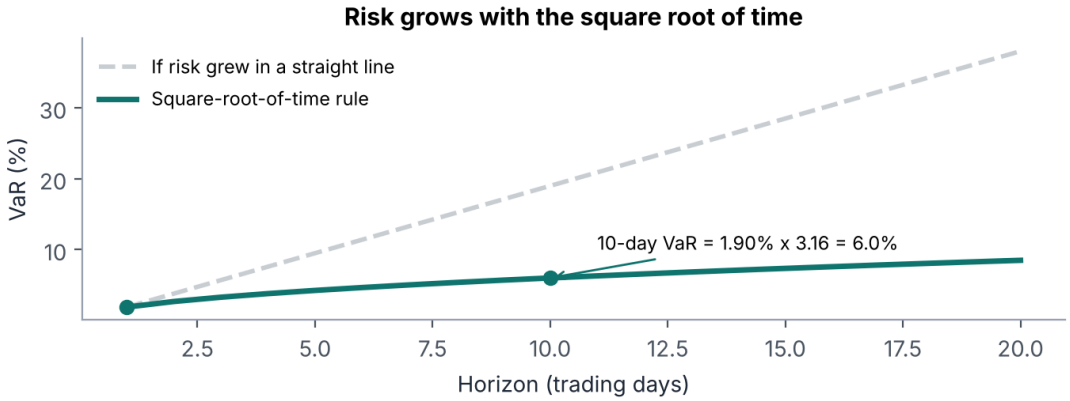


Figure 1. Your 1-day 99% VaR of 1.90% becomes about 6.0% over 10 days.

2. Correlation and diversification

Correlation measures how two assets move together, on a scale from −1 to +1. It is the single most important input for understanding a portfolio.

Correlation	Meaning	Example from your data
+1	Always move together	—
+0.5 to +0.7	Strongly linked (same sector)	Infosys & TCS: **0.61**; HDFC & ICICI: **0.52**
+0.2 to +0.4	Loosely linked (same market)	Most stock pairs: 0.2–0.4
about 0	Independent	Gold ETF vs any stock: **−0.03 to 0.01**
negative	Tend to move opposite	USD vs stocks: **−0.06 to −0.10**

Why correlation creates diversification

Portfolio risk is not the weighted average of each asset's risk. It is calculated with the **covariance matrix** Σ (every asset's volatility and every pair's correlation) and the weights *w*:

$$\text{portfolio volatility } \sigma_p = \sqrt{(\, w' \, \Sigma \, w \,)}$$

When correlations are below 1, the bad days of one asset are partly offset by normal or good days of another, so the total is **smaller than the sum of the parts**. The gap is the diversification benefit.

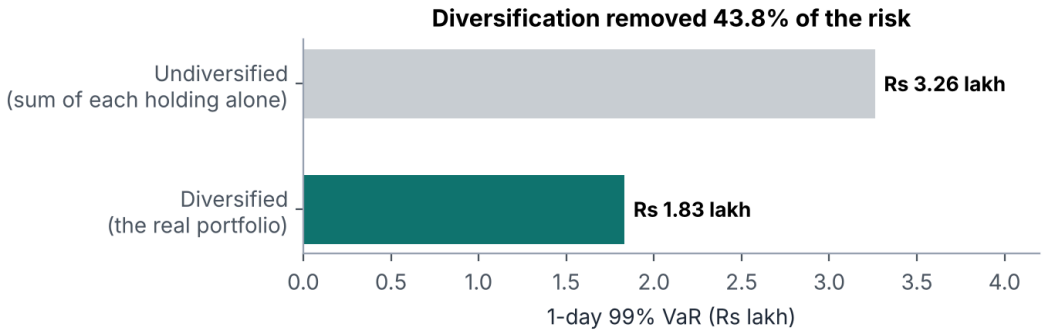


Figure 2. Your portfolio: the 10 holdings alone add up to Rs 3.26 lakh of VaR, but together only Rs 1.83 lakh.

Watch out: correlations rise in a crisis

Correlations are measured over normal times. In a crash, almost everything falls together and correlations jump towards 1, so diversification protects you **least when you need it most**. This is why stress tests (section 8) include an “everything falls” scenario.

3. Fat tails: why the bell curve lies

Many risk formulas assume returns follow a **normal distribution** (the bell curve). Real market returns do not.

Two statistics describe the shape of a distribution:

- **Skewness**: is the distribution lopsided? Negative skew means the big moves tend to be *down* (crashes are larger than rallies). Stock markets usually have negative skew.
- **Excess kurtosis**: how fat the tails are. A normal distribution has 0. Stock returns typically show 3 to 10 or more, which means extreme days happen far more often than the bell curve predicts.

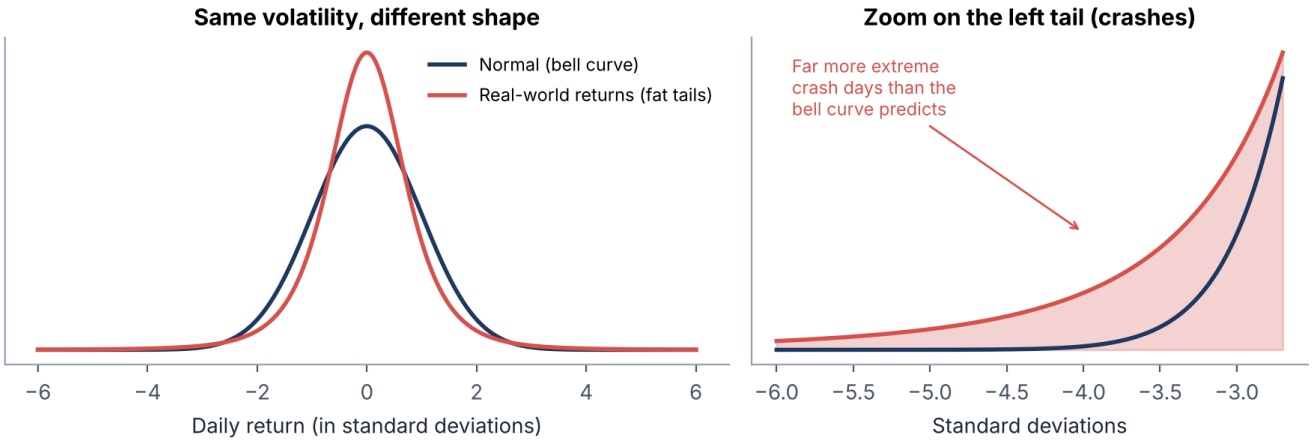


Figure 3. Real returns (red) have a taller, narrower middle AND fatter tails than a bell curve with the same volatility.

Finance insight: what fat tails do to risk numbers

Because real returns have **more calm days** and **more extreme days** than a bell curve, a model that assumes normality will:

- **overestimate** moderate risk (95% level), and
- **underestimate** extreme risk (99% level and beyond).

Your own results showed exactly this crossover in Step 2. It is the most important single idea in market risk.

4. Value at Risk (VaR)

VaR answers one question in one number: **"How much could I lose on a bad day?"**

> **1-day 99% VaR = Rs 1.90 lakh** means: on 99 out of 100 trading days, this portfolio should not lose more than Rs 1.90 lakh. On about 1 day in 100, it will lose more.

Every VaR number has three parts:

Part	Your project uses	Notes
Confidence level	95% and 99%	Banks use 99% for regulatory capital
Time horizon	1 day and 10 days	Basel uses 10 days: the time it could take to sell positions
Portfolio value	Rs 1 crore	Turns a % loss into rupees

Three ways to calculate VaR

Method	How it works	Strength	Weakness
Historical simulation	Sort all past daily returns and read off the worst 1% (or 5%)	No assumptions; uses real crashes	Assumes the future looks like the past; reacts slowly
Parametric (variance-covariance)	Assume a bell curve: VaR = − (mean + z × std)	Fast, simple, one formula	Ignores fat tails; underestimates extreme losses
Monte Carlo simulation	Simulate thousands of possible days using volatilities and correlations	Flexible; can use any distribution	Only as good as the distribution you choose

The **z-score** in the parametric formula is how many standard deviations below the average the cut-off sits: **−1.645** for 95% and **−2.326** for 99%.

> **What VaR does NOT tell you**
>
> VaR tells you where the worst 1% of days *begins*. It says nothing about **how bad** those days are. Two portfolios can have the same VaR, while one loses 2% on its worst days and the other loses 20%. That gap is why Expected Shortfall exists.

5. Expected Shortfall (ES)

Expected Shortfall (also called **CVaR**, conditional VaR) answers the follow-up question: **"When a loss does go beyond VaR, how much do I lose on average?"**

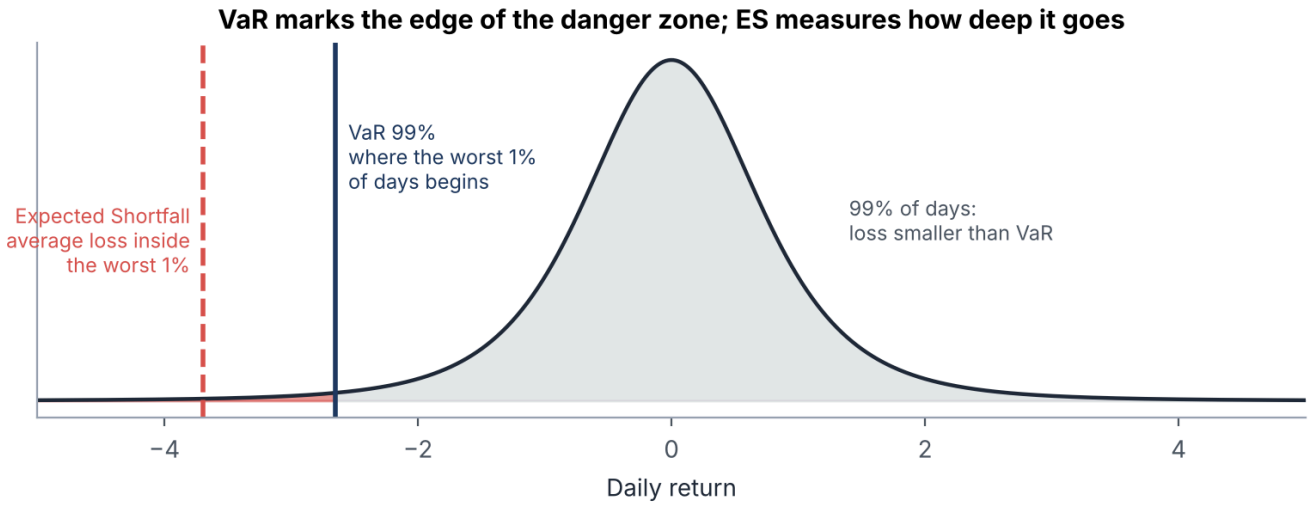


Figure 4. VaR is a line at the edge of the tail. ES is the average of everything beyond it, so it is always larger.

Under the Basel **FRTB** rules (the Fundamental Review of the Trading Book), banks measure trading risk with **ES at 97.5%** instead of VaR at 99%. The two give similar numbers for a bell curve, but ES grows much more when tails are fat, so it captures exactly the risk VaR misses.

	VaR	Expected Shortfall
Question answered	Where does the danger zone start?	How deep is the danger zone on average?
Sees tail size?	No	Yes
Regulatory standard	Basel II / 2.5 (99%)	Basel FRTB (97.5%)
Adds up nicely across desks?	Not always	Yes (it is "coherent")

6. Backtesting a risk model

A VaR number is a prediction. **Backtesting** checks whether those predictions came true.

Each day, calculate VaR using only data available up to the day before. Then compare it with what actually happened. A day where the real loss was bigger than VaR is an **exception** (or breach). A correct 99% model should have exceptions on about **1% of days**, roughly 2 to 3 per year.

Kupiec test (proportion of failures)

A statistical test of whether the number of exceptions is consistent with the confidence level. It produces a **p-value**: above 0.05 means the model passes; at or below 0.05 means it has too many (or too few) exceptions to be trusted.

Basel traffic light

Exceptions in last 250 days	Zone	What happens to a bank
0 – 4	Green	Model accepted
5 – 9	Yellow	Extra capital charge
10 or more	Red	Model rejected; must be fixed

Key idea: clustering

Exceptions should be scattered randomly. If they bunch together (several in one month), the model reacts too slowly to changing markets. This is tested with the **Christoffersen** independence test, and you can see it by eye on a backtest chart.

7. Risk decomposition

Knowing the total VaR is not enough. A portfolio manager needs to know **which holding is driving it**.

Measure	Question it answers
Standalone VaR	What would the VaR be if I held only this asset?
Marginal VaR	If I add a little more of this asset, how much does portfolio VaR rise?
Component VaR	How much of the total VaR belongs to this asset? (components add up exactly to the total)
Share of risk	Component VaR as a % of total VaR

The most useful comparison is **weight vs share of risk**. An asset whose share of risk is larger than its weight is a **risk concentrator**. An asset whose share is much smaller (or negative) is a **diversifier** or **hedge**.

8. Stress testing

VaR describes normal bad days. **Stress tests** ask what happens in a real crisis.

- **Historical scenarios** replay real events on today's portfolio: demonetisation (2016), the IL&FS crisis (2018), the COVID crash (2020), the Russia–Ukraine war (2022), election result day (2024).
- **Hypothetical scenarios** are “what if” shocks you design: banks fall 30%, the rupee crashes, everything falls at once.

Regulators require stress tests alongside VaR because VaR, by design, ignores the worst 1% of outcomes, and those are the outcomes that break firms.

9. How to analyse any portfolio: a checklist

You can apply this framework to any portfolio: a client's mutual funds, a company's treasury, or your own investments.

#	Question	Tool	What a good answer looks like
1	How much can it lose on a bad day?	VaR (95%, 99%)	A rupee number the owner can live with
2	How bad are the really bad days?	Expected Shortfall	ES not hugely above VaR (thin tails)
3	Does the risk model actually work?	Backtesting	~1% exceptions, Green zone, scattered breaches
4	Is it diversified?	Correlation matrix, diversified vs undiversified VaR	Low correlations; a large diversification benefit
5	Where is the risk concentrated?	Component VaR	No asset or sector with share of risk far above its weight
6	What breaks it?	Stress tests	Survivable losses in historical and hypothetical crises
7	What has hedging value?	Negative correlations, negative component VaR	Some assets that rise when stocks fall (gold, USD, bonds)

Advantages and risks of the portfolio in this project

Advantages

- Large, liquid blue-chip stocks across banking, IT, energy, consumer, infrastructure and telecom
- Gold has almost zero correlation with stocks: 12% of the money, only 1.9% of the risk
- USD is a natural hedge: its share of risk is negative (−0.4%)
- Diversification cuts VaR by 43.8%

Risks

- The 8 stocks are 78% of the capital but about 98% of the risk
- Banking concentration: HDFC + ICICI are 22% of the money but about 30% of the risk
- Fat tails: real crash days (up to −10.85%) are far bigger than VaR suggests
- No bonds: nothing to cushion a long equity bear market

PART 2

Building the project

Every file, every line, every command, and the output it produced on your laptop. Follow it in order and you can rebuild the whole project from an empty folder.

10. Setup: folders, VS Code and libraries

The finished project

```
VaR/
├── src/
│   ├── config.py               settings used by every file
│   ├── data_loader.py          Step 1: download prices, calculate returns
│   ├── var_basic.py            Step 2: historical and parametric VaR
│   ├── var_advanced.py         Step 3: Expected Shortfall and Monte Carlo
│   ├── backtest.py             Step 4: backtesting and EWMA
│   ├── risk_decomposition.py   Step 5: which asset drives the risk
│   ├── stress_test.py          Step 6: crises and what-if shocks
│   └── app.py                  Step 7: Streamlit dashboard
├── data/       created by Step 1 (prices.csv, returns.csv, portfolio_returns.csv)
└── outputs/    created by Steps 2-6 (tables .csv and charts .png)
```

One-time setup

1. Create a folder called **VaR** and open it in VS Code (*File → Open Folder*).
2. In the Explorer sidebar, create a folder named **src**. Every `.py` file goes inside it.
3. Open the terminal: *Terminal → New Terminal*. Its path must end in `\VaR>`, not `\src>`. If you are inside src, type `cd ..`
4. Check Python is installed, then install the libraries:

```
TERMINAL

PS C:\...\VaR> python --version
Python 3.13.14
```

```
TERMINAL

PS C:\...\VaR> pip install yfinance pandas numpy scipy matplotlib streamlit
...
Successfully installed ...
```

Library	Used for
yfinance	Downloading prices from Yahoo Finance
pandas	Tables of data (like Excel inside Python)
numpy	Fast maths: log, square root, matrix multiplication
scipy	Statistics: normal distribution, z-scores, chi-square test
matplotlib	Charts
streamlit	Turning a Python script into a web dashboard

How to add each new file

Right-click **src** → **New File** → type the exact name (e.g. `var_basic.py`) → paste the code → **Ctrl + S**. Always run commands from the VaR folder: `python src/<file>.py`. Ignore the yellow “new release of pip is available” notice.

STEP 1a

The settings file

`config.py` — run by the other files, never on its own

One place for every choice in the project. Every other file imports from here, so to change a stock, a weight or a confidence level, you edit only this file.

src/config.py

```python
"""
config.py - one place for all project settings.
Every other file imports from here, so you only change things once.
"""

# ---- Portfolio -----------------------------------------------------------
# Yahoo Finance tickers (.NS = NSE). INR=X is the USD/INR exchange rate.
PORTFOLIO = {
    "RELIANCE.NS":   0.12,   # Reliance Industries
    "HDFCBANK.NS":   0.12,   # HDFC Bank
    "ICICIBANK.NS":  0.10,   # ICICI Bank
    "INFY.NS":       0.10,   # Infosys
    "TCS.NS":        0.10,   # TCS
    "ITC.NS":        0.08,   # ITC
    "LT.NS":         0.08,   # Larsen & Toubro
    "BHARTIARTL.NS": 0.08,   # Bharti Airtel
    "GOLDBEES.NS":   0.12,   # Gold ETF
    "INR=X":         0.10,   # USD held (value moves with USD/INR)
}

PORTFOLIO_VALUE = 10_000_000      # Rs 1 crore notional

# ---- Data ----------------------------------------------------------------
START_DATE = "2015-01-01"
END_DATE = None                   # None = up to today

# ---- Risk settings -------------------------------------------------------
CONFIDENCE_LEVELS = [0.95, 0.99]
ES_CONFIDENCE = 0.975             # Basel FRTB standard for Expected Shortfall
HORIZONS = [1, 10]                # days
BACKTEST_WINDOW = 250             # ~1 trading year

# ---- Paths ---------------------------------------------------------------
DATA_DIR = "data"
OUTPUT_DIR = "outputs"

# Sanity check: weights must add up to 100%
assert abs(sum(PORTFOLIO.values()) - 1.0) < 1e-9, "Weights must sum to 1"
```

Code	What it does
`PORTFOLIO = {...}`	A **dictionary** of *ticker: weight* pairs. `.NS` = National Stock Exchange. `GOLDBEES.NS` = Nippon India Gold ETF. `INR=X` = the USD/INR exchange rate (holding dollars). 0.12 = 12% of the money.

Code	What it does
`PORTFOLIO_VALUE = 10_000_000`	Rs 1 crore notional (an assumed amount). Underscores only make the number readable. Used to turn % losses into rupees.
`START_DATE / END_DATE`	Download from 1 Jan 2015 to today (`None` = no end date). About 11 years, covering several crises.
`CONFIDENCE_LEVELS`	95% and 99% VaR.
`ES_CONFIDENCE = 0.975`	Expected Shortfall level set by Basel FRTB.
`HORIZONS = [1, 10]`	1-day and 10-day risk.
`BACKTEST_WINDOW = 250`	About one trading year; the look-back window for backtesting.
`DATA_DIR / OUTPUT_DIR`	Folder names for downloaded data and for results.
`assert abs(sum(...) - 1.0) < 1e-9`	Safety check: stops the program if the weights do not add up to 100%. `1e-9` is a tiny tolerance for rounding errors in decimals.

Why USD counts as an asset

Holding dollars is like owning an asset priced in rupees. When the rupee weakens (USD/INR goes from 83 to 85), the dollars are worth more. It usually rises when Indian stocks fall, which makes it a hedge.

STEP 1b

Download prices and calculate returns

data_loader.py — `python src/data_loader.py`

Downloads 11 years of daily prices for all 10 assets, cleans them, turns them into returns and saves three CSV files that every later step reads.

src/data_loader.py

```python
# data_loader.py - STEP 1: download prices and calculate returns
# Run from the VAR folder:  python src/data_loader.py

import os
import numpy as np
import pandas as pd
import yfinance as yf

from config import PORTFOLIO, PORTFOLIO_VALUE, START_DATE

tickers = list(PORTFOLIO.keys())
weights = pd.Series(PORTFOLIO)

# 1. Download daily closing prices (adjusted for dividends and splits)
print("Downloading prices...")
data = yf.download(tickers, start=START_DATE, auto_adjust=True)
prices = data["Close"]

# 2. Clean: fill small gaps with the previous day's price, drop rows still empty
prices = prices.ffill(limit=3).dropna()

# 3. Daily log returns = ln(today's price / yesterday's price)
returns = np.log(prices / prices.shift(1)).dropna()

# 3b. Remove bad data
# Large-cap stocks, gold and USD almost never move more than 20% in one day.
# A bigger jump is usually a stock split or a Yahoo data error, not a real move.
bad_days = (returns.abs() > 0.20).any(axis=1)
if bad_days.any():
    print("\nSuspicious days removed (data errors or splits):")
    print(returns[bad_days].round(3))
    returns = returns[~bad_days]

# 4. Portfolio daily return = sum of (weight x asset return)
portfolio_returns = (returns * weights).sum(axis=1)

# 5. Save to CSV files in a 'data' folder
os.makedirs("data", exist_ok=True)
prices.to_csv("data/prices.csv")
returns.to_csv("data/returns.csv")
portfolio_returns.to_csv("data/portfolio_returns.csv", header=["portfolio"])

# 6. Print results
print("\nData from", prices.index[0].date(), "to", prices.index[-1].date())
print("Number of trading days:", len(returns))

print("\nAnnual volatility of each asset (%):")
print((returns.std() * np.sqrt(252) * 100).round(2))

```

```
print("\nCorrelation between assets:")
print(returns.corr().round(2))

worst_day = portfolio_returns.min()
print("\n--- Portfolio ---")
print("Annual volatility (%):", round(portfolio_returns.std() * np.sqrt(252) * 100, 2))
print("Worst day (%):", round(worst_day * 100, 2), "on", portfolio_returns.idxmin().date())
print("Worst day loss (Rs):", round(worst_day * PORTFOLIO_VALUE))
print("Excess kurtosis:", round(portfolio_returns.kurt(), 2), "(above 0 = fat tails)")
```

Code	What it does
`import numpy as np ...`	Load the libraries. `as np` gives a short nickname.
`from config import ...`	Pull the settings from config.py. Both files must be in the same src folder.
`weights = pd.Series(PORTFOLIO)`	Turns the weight dictionary into a labelled column of numbers.
`yf.download(..., auto_adjust=True)`	Downloads daily prices. **auto_adjust** corrects for dividends and splits, so a 1:1 bonus issue does not look like a 50% crash.
`data["Close"]`	Keep only the closing price (Yahoo also gives Open, High, Low, Volume).
`prices.ffill(limit=3).dropna()`	USD/INR trades on days the NSE is closed and vice versa. **Forward-fill** copies the last known price over gaps of up to 3 days; **dropna** deletes any day that is still incomplete.
`np.log(prices / prices.shift(1))`	`shift(1)` moves the table down a row so each row holds yesterday's price. Today ÷ yesterday, then natural log = daily log return.
`bad_days = (returns.abs() > 0.20).any(axis=1)`	**Data-quality filter (step 3b).** Flags any day where any asset moved more than 20%. `axis=1` = check across the row.
`returns[~bad_days]`	Keep only the good days. `~` means “not”.
`(returns * weights).sum(axis=1)`	Portfolio return for each day = sum of weight × asset return. The same as Excel's SUMPRODUCT.
`os.makedirs("data", exist_ok=True)`	Create the data folder (no error if it already exists).
`.to_csv(...)`	Save prices, returns and portfolio returns as CSV files you can open in Excel.
`returns.std() * np.sqrt(252)`	Annual volatility of each asset.
`returns.corr()`	Correlation matrix of all 10 assets.
`portfolio_returns.idxmin()`	The date of the worst day.
`.kurt()`	Excess kurtosis: above 0 means fat tails.

Output

TERMINAL

```
PS C:\...\VaR> python src/data_loader.py
Downloading prices...

Suspicious days removed (data errors or splits):
   (a short table of the dates and assets that moved more than 20%)

Data from 2015-01-02 to 2026-11-06
Number of trading days: 3090

Annual volatility of each asset (%):
   (one line per asset)

Correlation between assets:
Ticker         BHARTIARTL.NS  GOLDBEES.NS  HDFCBANK.NS  ICICIBANK.NS  INFY.NS  INR=X  ITC.NS  LT.NS  RELIANCE.NS  TCS.NS
BHARTIARTL.NS           1.00        -0.02         0.28          0.29     0.20  -0.06    0.23   0.29         0.28    0.17
GOLDBEES.NS            -0.02         1.00         0.01         -0.01     0.01   0.00   -0.01   0.00        -0.03   -0.02
HDFCBANK.NS             0.28         0.01         1.00          0.52     0.27  -0.10    0.28   0.47         0.39    0.24
ICICIBANK.NS            0.29        -0.01         0.52          1.00     0.23  -0.10    0.29   0.50         0.36    0.20
INFY.NS                 0.20         0.01         0.27          0.23     1.00  -0.07    0.16   0.25         0.24    0.61
INR=X                  -0.06         0.00        -0.10         -0.10    -0.07   1.00   -0.07  -0.10        -0.07   -0.07
ITC.NS                  0.23        -0.01         0.28          0.29     0.16  -0.07    1.00   0.31         0.26    0.18
LT.NS                   0.29         0.00         0.47          0.50     0.25  -0.10    0.31   1.00         0.38    0.25
RELIANCE.NS             0.28        -0.03         0.39          0.36     0.24  -0.07    0.26   0.38         1.00    0.26
TCS.NS                  0.17        -0.02         0.24          0.20     0.61  -0.07    0.18   0.25         0.26    1.00

--- Portfolio ---
Worst day (%): -10.85 on 2020-03-23
...
```

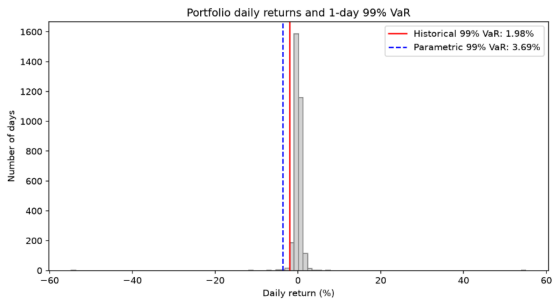
Abridged. The correlation matrix and worst day are from your run; the number of trading days is inferred from the backtest (2,840 tested + 250 window).

How to read it

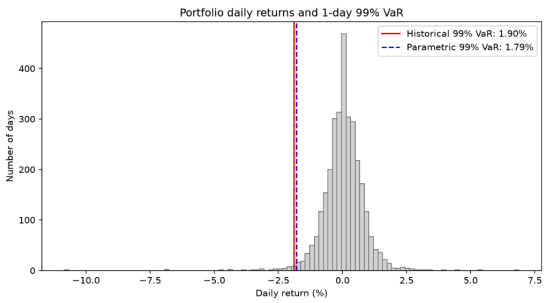
- **Same-sector pairs move together:** Infosys–TCS 0.61, HDFC–ICICI 0.52, L&T–ICICI 0.50.
- **Gold is independent** of every stock (about 0.00): a strong diversifier.
- **USD is slightly negative** against stocks (−0.06 to −0.10): when Indian stocks fall, the dollar tends to rise a little.

A real data-quality lesson

The first run, before the filter existed, showed the portfolio losing **54.72% on 19 Dec 2019** and an excess kurtosis of 888. A diversified large-cap portfolio never does that. One asset had a bad price in the Yahoo data (most likely an unadjusted split or a one-day glitch). The result was a histogram stretched from −60% to +60% and VaR numbers that made no sense.



Before the filter: one bad price stretches the chart to ±60%.



After the filter: a normal-looking return distribution.

Finance insight

Checking data before trusting a model is real risk-analyst work. Always look at the worst and best days and ask whether they make economic sense. Mention this as a **data quality checks** section in any report.

STEP 2

Historical and parametric VaR

var_basic.py — `python src/var_basic.py`

Your first real VaR numbers, calculated two ways at two confidence levels and two horizons.

src/var_basic.py

```python
# var_basic.py - STEP 2: Historical VaR and Parametric (Normal) VaR
# Run from the VAR folder:  python src/var_basic.py

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import norm

from config import PORTFOLIO_VALUE, CONFIDENCE_LEVELS, HORIZONS, OUTPUT_DIR

# 1. Load the portfolio daily returns saved in Step 1
returns = pd.read_csv("data/portfolio_returns.csv", index_col=0, parse_dates=True)["portfolio"]

mean = returns.mean()
std = returns.std()

results = []

for cl in CONFIDENCE_LEVELS:          # 0.95 and 0.99
    for days in HORIZONS:             # 1 and 10

        # 2. HISTORICAL VaR
        # Sort all past daily returns and pick the worst 5% (or 1%) cut-off point
        hist_1day = -np.percentile(returns, (1 - cl) * 100)

        # 3. PARAMETRIC (NORMAL) VaR
        # Assume returns follow a bell curve: VaR = -(mean + z * std)
        z = norm.ppf(1 - cl)          # z = -1.645 for 95%, -2.326 for 99%
        para_1day = -(mean + z * std)

        # 4. Scale from 1 day to N days with the square-root-of-time rule
        hist_var = hist_1day * np.sqrt(days)
        para_var = para_1day * np.sqrt(days)

        results.append({
            "Confidence": f"{int(cl * 100)}%",
            "Days": days,
            "Historical VaR %": round(hist_var * 100, 2),
            "Historical VaR Rs": round(hist_var * PORTFOLIO_VALUE),
            "Parametric VaR %": round(para_var * 100, 2),
            "Parametric VaR Rs": round(para_var * PORTFOLIO_VALUE),
        })

table = pd.DataFrame(results)

# 5. Print and save the results
pd.set_option("display.width", 140)
print("\nValue at Risk on a Rs 1 crore portfolio")
print(table.to_string(index=False))
```

```python

os.makedirs(OUTPUT_DIR, exist_ok=True)
table.to_csv(f"{OUTPUT_DIR}/var_basic.csv", index=False)

# 6. Chart: histogram of daily returns with the 99% VaR lines
hist_99 = -np.percentile(returns, 1)
para_99 = -(mean + norm.ppf(0.01) * std)

plt.figure(figsize=(10, 5))
plt.hist(returns * 100, bins=100, color="lightgray", edgecolor="gray")
plt.axvline(-hist_99 * 100, color="red", label=f"Historical 99% VaR: {hist_99*100:.2f}%")
plt.axvline(-para_99 * 100, color="blue", linestyle="--", label=f"Parametric 99% VaR: {para_99*100:.2f}%")
plt.title("Portfolio daily returns and 1-day 99% VaR")
plt.xlabel("Daily return (%)")
plt.ylabel("Number of days")
plt.legend()
plt.savefig(f"{OUTPUT_DIR}/var_histogram.png", dpi=150)
plt.show()

print(f"\nSaved table and chart in '{OUTPUT_DIR}/'")
```

Code	What it does
`pd.read_csv("data/portfolio_returns.csv", ...)`	Load the portfolio returns saved in Step 1. `index_col=0, parse_dates=True` makes the date column the row label.
`for cl in CONFIDENCE_LEVELS: for days in HORIZONS:`	Two loops: repeat the calculation for every combination (95%/99% × 1/10 days) = 4 rows.
`-np.percentile(returns, (1 - cl) * 100)`	**Historical VaR.** For 99%, find the return that only 1% of days were worse than. The minus sign turns a negative return into a positive loss.
`norm.ppf(1 - cl)`	The z-score: −1.645 for 95%, −2.326 for 99%.
`-(mean + z * std)`	**Parametric VaR** using the bell-curve formula.
`* np.sqrt(days)`	Square-root-of-time scaling from 1 day to 10 days.
`results.append({...})`	Add one row to the results list.
`pd.DataFrame(results)`	Turn the list of rows into a table.
`plt.hist(...) / plt.axvline(...)`	Histogram of daily returns with vertical lines at both 99% VaRs.
`plt.savefig(...) / plt.show()`	Save the chart to outputs, then open it in a window. **Close the window** for the script to finish.

Output

```
TERMINAL

PS C:\...\VaR> python src/var_basic.py
Value at Risk on a Rs 1 crore portfolio
Confidence  Days  Historical VaR %  Historical VaR Rs  Parametric VaR %  Parametric VaR Rs
       95%     1             1.10             110427              1.25             125382
       95%    10             3.49             349202              3.96             396491
       99%     1             1.90             190491              1.79             179091
       99%    10             6.02             602385              5.66             566334

Saved table and chart in 'outputs/'
```

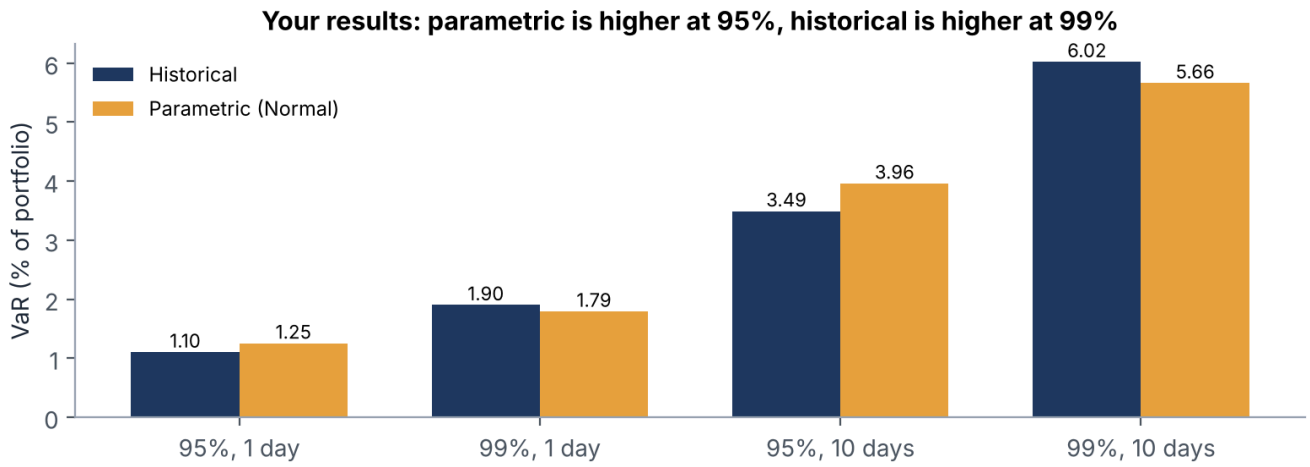


Figure 5. Your VaR results. Note how the two methods swap places between 95% and 99%.

How to read it

- **Headline:** on 99 out of 100 days, this Rs 1 crore portfolio should not lose more than **Rs 1.90 lakh** in a day.
- **At 95%, parametric is higher** (1.25% vs 1.10%). **At 99%, historical is higher** (1.90% vs 1.79%). This is the fat-tails crossover from section 3, reproduced by your own data.
- The 10-day numbers are the 1-day numbers × 3.16.

STEP 3

Expected Shortfall and Monte Carlo VaR

`var_advanced.py` — `python src/var_advanced.py`

Adds the tail-depth measure regulators now use (ES at 97.5%) and two simulation methods, one with bell-curve tails and one with fat tails.

src/var_advanced.py

```
# var_advanced.py - STEP 3: Expected Shortfall and Monte Carlo VaR
# Run from the VAR folder:  python src/var_advanced.py

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import norm

from config import PORTFOLIO, PORTFOLIO_VALUE, ES_CONFIDENCE, OUTPUT_DIR

# 1. Load data from Step 1
asset_returns = pd.read_csv("data/returns.csv", index_col=0, parse_dates=True)
port_returns = pd.read_csv("data/portfolio_returns.csv", index_col=0, parse_dates=True)
["portfolio"]

weights = pd.Series(PORTFOLIO)[asset_returns.columns]   # same order as the columns


# Helper: VaR and Expected Shortfall from any list of returns
def var_and_es(returns, cl):
    cutoff = np.percentile(returns, (1 - cl) * 100)      # the VaR point
    var = -cutoff
    es = -returns[returns <= cutoff].mean()              # average of the days beyond VaR
    return var, es


# 2. HISTORICAL - real past days
hist = port_returns.values

# 3. PARAMETRIC (NORMAL) - formula for a bell curve
mean, std = port_returns.mean(), port_returns.std()

def normal_var_es(cl):
    z = norm.ppf(1 - cl)
    var = -(mean + z * std)
    es = -(mean - std * norm.pdf(z) / (1 - cl))
    return var, es

# 4. MONTE CARLO - simulate 10,000 possible days
np.random.seed(42)            # same random numbers every run, so results repeat
n_sims = 10000

asset_mean = asset_returns.mean().values
asset_cov = asset_returns.cov().values      # volatilities + correlations of all assets

# 4a. Normal Monte Carlo: random days drawn from a multivariate bell curve
sim_normal = np.random.multivariate_normal(asset_mean, asset_cov, n_sims)
mc_normal = sim_normal @ weights.values     # portfolio return for each simulated day
```

```python

# 4b. Student-t Monte Carlo: same correlations, but fat tails (df = 4)
df = 4
shock = np.random.multivariate_normal(np.zeros(len(weights)), asset_cov, n_sims)
scale = np.sqrt(np.random.chisquare(df, n_sims) / (df - 2))   # makes some days extreme
sim_t = asset_mean + shock / scale[:, None]
mc_t = sim_t @ weights.values

# 5. Results table
methods = {
    "Historical": lambda cl: var_and_es(hist, cl),
    "Parametric (Normal)": normal_var_es,
    "Monte Carlo (Normal)": lambda cl: var_and_es(mc_normal, cl),
    "Monte Carlo (Student-t)": lambda cl: var_and_es(mc_t, cl),
}

rows = []
for name, calc in methods.items():
    var95, _ = calc(0.95)
    var99, _ = calc(0.99)
    _, es975 = calc(ES_CONFIDENCE)
    rows.append({
        "Method": name,
        "VaR 95% (%)": round(var95 * 100, 2),
        "VaR 99% (%)": round(var99 * 100, 2),
        "ES 97.5% (%)": round(es975 * 100, 2),
        "VaR 99% (Rs)": round(var99 * PORTFOLIO_VALUE),
        "ES 97.5% (Rs)": round(es975 * PORTFOLIO_VALUE),
    })

table = pd.DataFrame(rows)
pd.set_option("display.width", 160)
print("\n1-day VaR and Expected Shortfall on a Rs 1 crore portfolio")
print(table.to_string(index=False))

os.makedirs(OUTPUT_DIR, exist_ok=True)
table.to_csv(f"{OUTPUT_DIR}/var_advanced.csv", index=False)

# 6. Chart: historical returns with VaR 99% and ES 97.5%
var99, _ = var_and_es(hist, 0.99)
_, es975 = var_and_es(hist, ES_CONFIDENCE)

plt.figure(figsize=(10, 5))
plt.hist(hist * 100, bins=100, color="lightgray", edgecolor="gray")
plt.axvline(-var99 * 100, color="red", label=f"Historical VaR 99%: {var99*100:.2f}%")
plt.axvline(-es975 * 100, color="purple", linestyle="--", label=f"Historical ES 97.5%:
{es975*100:.2f}%")
plt.title("VaR shows where the tail starts; ES shows the average loss inside it")
plt.xlabel("Daily return (%)")
plt.ylabel("Number of days")
plt.legend()
plt.savefig(f"{OUTPUT_DIR}/var_es_chart.png", dpi=150)
plt.show()

print(f"\nSaved table and chart in '{OUTPUT_DIR}/'")
```

Code	What it does
`def var_and_es(returns, cl):`	A reusable **function**: give it any set of returns and a confidence level, and it returns VaR and ES.

Code	What it does
`returns[returns <= cutoff].mean()`	**Expected Shortfall**: keep only the days worse than the VaR cut-off and average them.
`mean - std * norm.pdf(z) / (1 - cl)`	The bell-curve formula for ES (you do not need to memorise it).
`np.random.seed(42)`	Fix the random numbers so every run gives the same result. Essential for reproducible reports.
`asset_returns.cov()`	The **covariance matrix**: every asset's volatility and every pair's correlation in one table.
`np.random.multivariate_normal(mean, cov, 10000)`	Simulate 10,000 possible days for all 10 assets at once, keeping their correlations. When HDFC has a bad simulated day, ICICI usually does too.
`sim_normal @ weights.values`	`@` is matrix multiplication: turns each simulated day into one portfolio return (SUMPRODUCT for 10,000 rows at once).
`np.random.chisquare(df, n) / (df - 2)`	The **Student-t** trick: on some days this scale is small, and dividing by it makes that day extreme. df = 4 gives realistic fat tails.
`methods = {"Historical": lambda cl: ...}`	A dictionary of the four methods. `lambda` is a one-line mini-function, so every method can be called the same way.

Output

TERMINAL

```
PS C:\...\VaR> python src/var_advanced.py
1-day VaR and Expected Shortfall on a Rs 1 crore portfolio
                Method  VaR 95% (%)  VaR 99% (%)  ES 97.5% (%)  VaR 99% (Rs)  ES 97.5% (Rs)
            Historical         1.10         1.90           ...        190491            ...
    Parametric (Normal)         1.25         1.79          1.80        179091         180000
   Monte Carlo (Normal)         1.29         1.81          1.84        181371         183677
Monte Carlo (Student-t)         1.14         2.03          2.14        202609         213602

Saved table and chart in 'outputs/'
```

The two Monte Carlo rows are from your run. The historical and parametric VaR values match Step 2; parametric ES (about 1.80%) is calculated from them. Fill in historical ES from your own output.

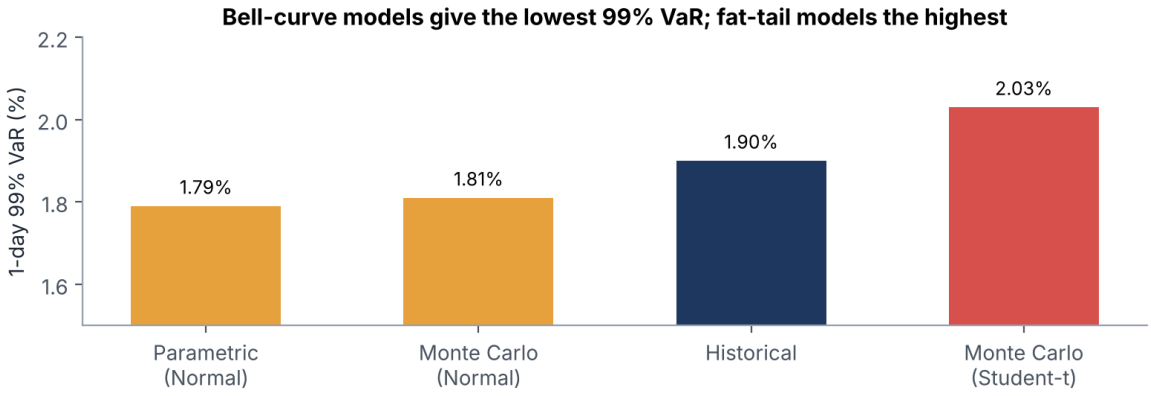


Figure 6. 1-day 99% VaR by method: the more a model allows for fat tails, the higher the number.

How to read it

- **Normal Monte Carlo ≈ parametric** (1.81% vs 1.79%): both assume bell curves, so they agree.
- **Student-t Monte Carlo** is *lower* at 95% (1.14%) but *higher* at 99% (2.03%) and in ES (2.14%). Same crossover again: fat tails mean more calm days and more extreme days.
- The Student-t 99% VaR (2.03%) sits closest to the historical 1.90%, so it describes reality better than the normal models.

STEP 4

Backtesting: would the models have worked?

backtest.py — `python src/backtest.py`

Replays 11 years day by day. Each day, VaR is calculated from the previous 250 days only, then compared with the next day's real return. Three models are tested, including a new one: EWMA.

src/backtest.py

```python
# backtest.py - STEP 4: Backtesting - would our VaR models have worked?
# Run from the VAR folder:  python src/backtest.py
#
# Idea: every day, calculate 99% VaR using ONLY the past 250 days,
# then check if the next day's real loss was bigger (an "exception").
# A good 99% model should have about 1 exception per 100 days.

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import norm, chi2

from config import BACKTEST_WINDOW, OUTPUT_DIR

returns = pd.read_csv("data/portfolio_returns.csv", index_col=0, parse_dates=True)["portfolio"]

cl = 0.99
window = BACKTEST_WINDOW          # 250 days
z = norm.ppf(1 - cl)              # -2.326

# 1. Historical VaR: 1st percentile of the past 250 days
hist_var = -returns.rolling(window).quantile(1 - cl).shift(1)

# 2. Parametric VaR: bell curve fitted to the past 250 days
roll_mean = returns.rolling(window).mean()
roll_std = returns.rolling(window).std()
para_var = -(roll_mean + z * roll_std).shift(1)

# 3. EWMA VaR (RiskMetrics): recent days count more than old days
# lambda = 0.94 means each day's weight is 94% of the next day's weight
lam = 0.94
ewma_std = np.sqrt((returns ** 2).ewm(alpha=1 - lam).mean())
ewma_var = -(z * ewma_std).shift(1)

# .shift(1) above = VaR for today uses data only up to yesterday (no peeking!)

models = {"Historical": hist_var, "Parametric": para_var, "EWMA": ewma_var}


# Kupiec test: is the number of exceptions statistically OK?
def kupiec_pvalue(exceptions, days, p=1 - cl):
    x, T = exceptions, days
    if x == 0:
        lr = -2 * T * np.log(1 - p)
    else:
        rate = x / T
        lr = -2 * ((T - x) * np.log(1 - p) + x * np.log(p)
                   - (T - x) * np.log(1 - rate) - x * np.log(rate))
```

```python
    return 1 - chi2.cdf(lr, df=1)


# Basel traffic light (for 250 days at 99%)
def basel_zone(exceptions):
    if exceptions <= 4:
        return "Green"
    elif exceptions <= 9:
        return "Yellow"
    return "Red"


# 4. Test every model
rows = []
for name, var in models.items():
    test = pd.DataFrame({"actual": returns, "var": var}).dropna()
    exceptions = test["actual"] < -test["var"]          # loss bigger than VaR

    days = len(test)
    n_exc = exceptions.sum()
    last_250 = exceptions.iloc[-250:].sum()
    pval = kupiec_pvalue(n_exc, days)

    rows.append({
        "Model": name,
        "Days tested": days,
        "Exceptions": n_exc,
        "Expected": round(days * (1 - cl)),
        "Exception rate %": round(n_exc / days * 100, 2),
        "Kupiec p-value": round(pval, 3),
        "Kupiec result": "Pass" if pval > 0.05 else "Fail",
        "Last 250 days": last_250,
        "Basel zone": basel_zone(last_250),
    })

table = pd.DataFrame(rows)
pd.set_option("display.width", 160)
print("\nBacktest of 1-day 99% VaR")
print(table.to_string(index=False))

os.makedirs(OUTPUT_DIR, exist_ok=True)
table.to_csv(f"{OUTPUT_DIR}/backtest.csv", index=False)

# 5. Chart: real returns vs each model's VaR line, exceptions marked in red
fig, axes = plt.subplots(3, 1, figsize=(12, 10), sharex=True)
for ax, (name, var) in zip(axes, models.items()):
    test = pd.DataFrame({"actual": returns, "var": var}).dropna()
    exc = test[test["actual"] < -test["var"]]
    ax.plot(test.index, test["actual"] * 100, color="gray", linewidth=0.5, label="Daily return")
    ax.plot(test.index, -test["var"] * 100, color="blue", linewidth=1, label="-VaR 99%")
    ax.scatter(exc.index, exc["actual"] * 100, color="red", s=12, label=f"Exceptions
({len(exc)})")
    ax.set_title(f"{name} VaR")
    ax.set_ylabel("Return (%)")
    ax.legend(loc="lower left")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/backtest_chart.png", dpi=150)
plt.show()

print(f"\nSaved table and chart in '{OUTPUT_DIR}/'")
```

Code	What it does
`returns.rolling(window).quantile(1 - cl)`	A moving window: for every day, the 1st percentile of the previous 250 days (historical VaR).
`.shift(1)`	**No peeking.** Today's VaR may only use data up to yesterday. Without this, the backtest would cheat and look too good.
`(returns ** 2).ewm(alpha=1 - lam).mean()`	**EWMA volatility** (RiskMetrics, J.P. Morgan 1994). With λ = 0.94, each older day counts 94% as much as the next, so recent days dominate and VaR reacts quickly.
`test["actual"] < -test["var"]`	True on days the loss was bigger than VaR: an exception.
`def kupiec_pvalue(...)`	The Kupiec likelihood-ratio test, compared with a chi-square distribution (1 degree of freedom). p > 0.05 = pass.
`def basel_zone(...)`	The traffic light: 0–4 Green, 5–9 Yellow, 10+ Red.
`exceptions.iloc[-250:].sum()`	Exceptions in the most recent 250 days, the window regulators look at.
`plt.subplots(3, 1, sharex=True)`	Three charts stacked, sharing the same date axis.

Output

TERMINAL

```
PS C:\...\VaR> python src/backtest.py
Backtest of 1-day 99% VaR
     Model  Days tested  Exceptions  Expected  Exception rate %  Kupiec p-value Kupiec result  Last 250 days Basel zone
Historical         2840          44        28              1.55           0.006          Fail              7     Yellow
Parametric         2840          50        28              1.76           0.000          Fail              9     Yellow
      EWMA         3089          59        31              1.91           0.000          Fail              5     Yellow

Saved table and chart in 'outputs/'
```

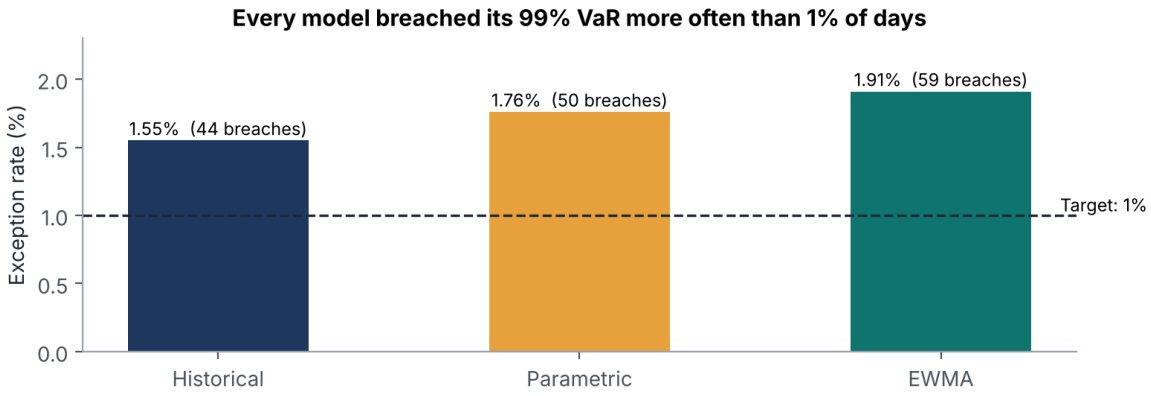


Figure 7. Exception rates: all three models breach almost twice as often as a correct 99% model.

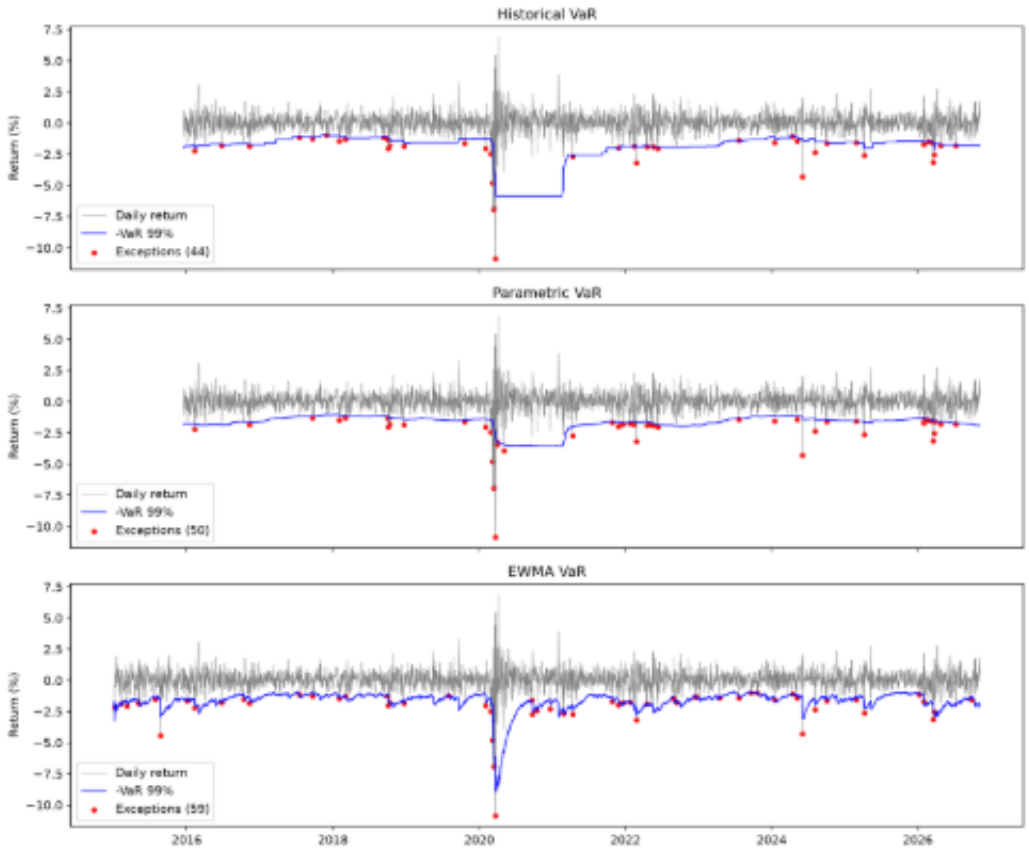


Figure 8. Your backtest chart. Blue = −VaR, grey = daily returns, red = exceptions. Top: historical; middle: parametric; bottom: EWMA.

How to read it

- **Every model fails Kupiec and sits in the Yellow zone.** A bank using any of them would pay an extra capital charge. The root cause is fat tails.
- **The ghost effect** (top panel): after March 2020, historical VaR drops to about −5% and stays flat for a year, then jumps back overnight. The crash days were inside the 250-day window for exactly a year; when they left it, VaR changed even though the market did not.
- **EWMA reacts fastest** (bottom panel): it plunges at the crash and recovers smoothly. It has the fewest recent exceptions (5) but the most overall (59), because in calm periods it shrinks VaR quickly and is then caught by sudden shocks.
- **Exceptions cluster** around early 2020: models with long windows react too slowly when a crisis begins.

STEP 5

Risk decomposition: who drives the risk?

`risk_decomposition.py` — `python src/risk_decomposition.py`

Splits the portfolio VaR into each holding's contribution and measures the diversification benefit in rupees.

src/risk_decomposition.py

```python
# risk_decomposition.py - STEP 5: Which asset adds the most risk?
# Run from the VAR folder:  python src/risk_decomposition.py

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import norm

from config import PORTFOLIO, PORTFOLIO_VALUE, OUTPUT_DIR

returns = pd.read_csv("data/returns.csv", index_col=0, parse_dates=True)
weights = pd.Series(PORTFOLIO)[returns.columns]

cl = 0.99
z = -norm.ppf(1 - cl)                # 2.326

cov = returns.cov()                  # covariance matrix (volatility + correlation)

# 1. Portfolio volatility = sqrt(w' x Cov x w)
port_std = np.sqrt(weights @ cov @ weights)
port_var = z * port_std              # diversified 99% VaR (in %)

# 2. Marginal VaR: how much VaR changes if we add a little more of this asset
marginal_var = z * (cov @ weights) / port_std

# 3. Component VaR: each asset's share of the total VaR (they add up exactly to the total)
component_var = weights * marginal_var

# 4. Standalone VaR: the VaR of each holding on its own (ignoring the others)
standalone_var = z * weights * returns.std()

table = pd.DataFrame({
    "Weight %": (weights * 100).round(1),
    "Standalone VaR (Rs)": (standalone_var * PORTFOLIO_VALUE).round(0),
    "Component VaR (Rs)": (component_var * PORTFOLIO_VALUE).round(0),
    "Share of risk %": (component_var / port_var * 100).round(1),
}).sort_values("Share of risk %", ascending=False)

undiversified = standalone_var.sum()

pd.set_option("display.width", 140)
print("\nRisk contribution of each asset (1-day 99% parametric VaR)")
print(table.to_string())
print("\nUndiversified VaR (sum of standalone):", round(undiversified * PORTFOLIO_VALUE), "Rs")
print("Diversified VaR (whole portfolio)     :", round(port_var * PORTFOLIO_VALUE), "Rs")
print("Diversification benefit              :", round((undiversified - port_var) *
PORTFOLIO_VALUE), "Rs",
      f"({(1 - port_var / undiversified) * 100:.1f}% less risk)")
```

```python

os.makedirs(OUTPUT_DIR, exist_ok=True)
table.to_csv(f"{OUTPUT_DIR}/risk_decomposition.csv")

# 5. Chart: weight vs share of risk for each asset
chart = table[["Weight %", "Share of risk %"]].sort_values("Share of risk %")
chart.plot(kind="barh", figsize=(10, 6), color=["lightgray", "crimson"])
plt.axvline(0, color="black", linewidth=0.8)
plt.title("Weight vs share of portfolio risk (99% VaR)")
plt.xlabel("%")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/risk_decomposition.png", dpi=150)
plt.show()

print(f"\nSaved table and chart in '{OUTPUT_DIR}/'")
```

Code	What it does
`z = -norm.ppf(1 - cl)`	2.326 for 99%.
`np.sqrt(weights @ cov @ weights)`	Portfolio volatility with the textbook matrix formula $\sigma_p = \sqrt{w'\Sigma w}$.
`z * (cov @ weights) / port_std`	**Marginal VaR**: how fast portfolio VaR rises as you add more of each asset.
`weights * marginal_var`	**Component VaR**: weight × marginal. The components add up exactly to the portfolio VaR.
`z * weights * returns.std()`	**Standalone VaR**: each holding's VaR on its own.
`standalone_var.sum()`	**Undiversified VaR**: what VaR would be if everything moved perfectly together.
`.sort_values("Share of risk %", ascending=False)`	Biggest risk contributors first.

Output

TERMINAL

```
PS C:\...\VaR> python src/risk_decomposition.py
Risk contribution of each asset (1-day 99% parametric VaR)
              Weight %  Standalone VaR (Rs)  Component VaR (Rs)  Share of risk %
RELIANCE.NS       12.0                  ...                 ...           ~16.7
ICICIBANK.NS      10.0                  ...                 ...           ~16.0
HDFCBANK.NS       12.0                  ...                 ...           ~14.5
INFY.NS           10.0                  ...                 ...           ~12.5
LT.NS              8.0                  ...                 ...           ~11.1
TCS.NS            10.0              34148.0             19487.0            10.6
BHARTIARTL.NS      8.0              32740.0             17414.0             9.5
ITC.NS             8.0              27971.0             13969.0             7.6
GOLDBEES.NS       12.0              26698.0              3488.0             1.9
INR=X             10.0               8730.0              -672.0            -0.4

Undiversified VaR (sum of standalone): 326188 Rs
Diversified VaR (whole portfolio)     : 183341 Rs
Diversification benefit               : 142847 Rs (43.8% less risk)
```

The top five rows scrolled off your terminal; their shares (~) are read from your chart.

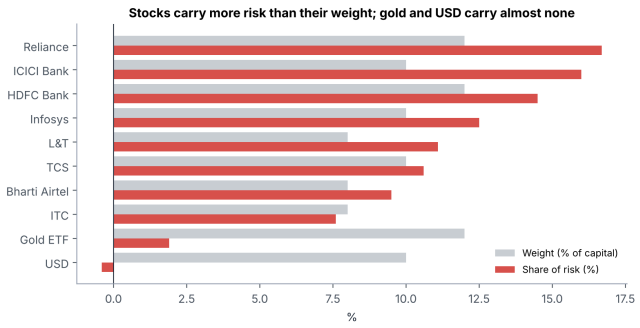


Figure 9. Weight vs share of risk.

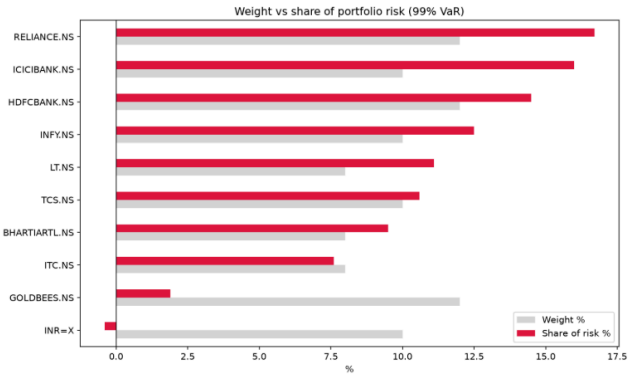


Figure 10. The same chart from your run.

How to read it

- **Stocks: 78% of the capital, about 98% of the risk.** Gold and USD: 22% of the capital, about 1.5% of the risk.
- **ICICI Bank** carries 1.6× its weight in risk. **HDFC + ICICI** together: 22% of capital, about 30% of risk, a sector concentration.
- **ITC** is the only stock with less risk than its weight: a defensive stock.
- **USD has negative component VaR**: adding it actually lowers total risk. That is the definition of a hedge.
- **Practical takeaway:** trimming a bank and adding gold would cut risk without giving up much return.

STEP 6

Stress testing: crises and what-ifs

stress_test.py — `python src/stress_test.py`

Replays five real Indian market crises on your portfolio and applies five hypothetical shocks, then compares every loss with VaR.

src/stress_test.py

```python
# stress_test.py - STEP 6: Stress testing - what happens in a crisis?
# Run from the VAR folder:  python src/stress_test.py
#
# VaR covers "normal bad days". Stress tests ask: "what if a real crisis happens?"

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from config import PORTFOLIO, PORTFOLIO_VALUE, OUTPUT_DIR

returns = pd.read_csv("data/returns.csv", index_col=0, parse_dates=True)
port_returns = pd.read_csv("data/portfolio_returns.csv", index_col=0, parse_dates=True)
["portfolio"]
weights = pd.Series(PORTFOLIO)[returns.columns]

# VaR numbers to compare against (historical, 99%)
var_1day = -np.percentile(port_returns, 1)
var_10day = var_1day * np.sqrt(10)

results = []

# ---------------------------------------------------------------------------
# PART A: HISTORICAL SCENARIOS - replay real Indian market crises
# (start and end dates are approximate market peak -> bottom)
# ---------------------------------------------------------------------------
historical_scenarios = {
    "Demonetisation (Nov-Dec 2016)":   ("2016-11-08", "2016-12-26"),
    "IL&FS crisis (Aug-Oct 2018)":     ("2018-08-28", "2018-10-26"),
    "COVID crash (Feb-Mar 2020)":      ("2020-02-20", "2020-03-23"),
    "Russia-Ukraine (Jan-Mar 2022)":   ("2022-01-18", "2022-03-07"),
    "Election result day (4 Jun 2024)": ("2024-06-04", "2024-06-04"),
}

for name, (start, end) in historical_scenarios.items():
    period = returns.loc[start:end]
    if len(period) == 0:
        continue                                  # dates not in our data
    asset_change = np.exp(period.sum()) - 1       # total % change of each asset
    portfolio_change = (weights * asset_change).sum()
    results.append({"Scenario": name, "Type": "Historical",
                    "Days": len(period), "Portfolio change %": portfolio_change * 100})

# ---------------------------------------------------------------------------
# PART B: HYPOTHETICAL SCENARIOS - "what if" shocks we choose ourselves
# ---------------------------------------------------------------------------
stocks = [t for t in weights.index if t not in ("GOLDBEES.NS", "INR=X")]
banks = ["HDFCBANK.NS", "ICICIBANK.NS"]
```

```python
it = ["INFY.NS", "TCS.NS"]

hypothetical_scenarios = {
    # shock = % move for each asset (anything not listed = 0%)
    "Market crash: stocks -20%, gold +5%, USD +3%":
        {**{s: -0.20 for s in stocks}, "GOLDBEES.NS": 0.05, "INR=X": 0.03},
    "Banking crisis: banks -30%, other stocks -10%":
        {**{s: -0.10 for s in stocks}, **{b: -0.30 for b in banks}},
    "IT sell-off: Infosys & TCS -25%":
        {i: -0.25 for i in it},
    "Rupee crash: USD +10%, stocks -10%":
        {**{s: -0.10 for s in stocks}, "INR=X": 0.10},
    "Everything falls: stocks -15%, gold -10%, USD -5%":
        {**{s: -0.15 for s in stocks}, "GOLDBEES.NS": -0.10, "INR=X": -0.05},
}

for name, shock in hypothetical_scenarios.items():
    asset_change = pd.Series(shock).reindex(weights.index, fill_value=0)
    portfolio_change = (weights * asset_change).sum()
    results.append({"Scenario": name, "Type": "Hypothetical",
                    "Days": "-", "Portfolio change %": portfolio_change * 100})

# ------------------------------------------------------------------------
# Results
# ------------------------------------------------------------------------
table = pd.DataFrame(results)
table["P&L (Rs)"] = (table["Portfolio change %"] / 100 * PORTFOLIO_VALUE).round(0)
table["Loss / 10-day VaR"] = (-table["Portfolio change %"] / 100 / var_10day).round(1)
table["Portfolio change %"] = table["Portfolio change %"].round(2)

pd.set_option("display.width", 160)
print("\nStress test results on a Rs 1 crore portfolio")
print(table.to_string(index=False))
print(f"\nFor comparison: 1-day 99% VaR = Rs {var_1day * PORTFOLIO_VALUE:,.0f}, "
      f"10-day 99% VaR = Rs {var_10day * PORTFOLIO_VALUE:,.0f}")
print("'Loss / 10-day VaR' above 1 means the crisis loss was bigger than VaR predicted.")

# Worst 5 single days in our data
worst = port_returns.nsmallest(5)
print("\nWorst 5 days for this portfolio:")
for date, r in worst.items():
    print(f"  {date.date()}   {r * 100:6.2f}%   Rs {r * PORTFOLIO_VALUE:,.0f}")

os.makedirs(OUTPUT_DIR, exist_ok=True)
table.to_csv(f"{OUTPUT_DIR}/stress_test.csv", index=False)

# Chart: P&L of each scenario with the VaR lines
chart = table.sort_values("P&L (Rs)")
colors = ["crimson" if t == "Historical" else "darkorange" for t in chart["Type"]]
plt.figure(figsize=(11, 6))
plt.barh(chart["Scenario"], chart["P&L (Rs)"] / 100000, color=colors)
plt.axvline(-var_1day * PORTFOLIO_VALUE / 100000, color="blue", linestyle="--", label="1-day 99% VaR")
plt.axvline(-var_10day * PORTFOLIO_VALUE / 100000, color="navy", label="10-day 99% VaR")
plt.axvline(0, color="black", linewidth=0.8)
plt.title("Stress test P&L (red = historical, orange = hypothetical)")
plt.xlabel("Profit / loss (Rs lakh)")
plt.legend()
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/stress_test.png", dpi=150)
plt.show()

print(f"\nSaved table and chart in '{OUTPUT_DIR}/'")
```

Code	What it does
`historical_scenarios = {...}`	Name → (start date, end date), roughly market peak to bottom.
`returns.loc[start:end]`	Select only the days inside the crisis window.
`np.exp(period.sum()) - 1`	Add up the daily log returns over the period, then convert back to an ordinary % change (e.g. −0.30 → −26%).
`{**{s: -0.20 for s in stocks}, ...}`	Builds a shock dictionary: every stock −20%, then adds the gold and USD moves. `**` merges dictionaries.
`.reindex(weights.index, fill_value=0)`	Any asset not mentioned in a scenario gets a 0% move.
`"Loss / 10-day VaR"`	Above 1 = the crisis loss was bigger than the 10-day VaR predicted.
`port_returns.nsmallest(5)`	The five worst single days in the data.

Output

TERMINAL

```
PS C:\...\VaR> python src/stress_test.py
Stress test results on a Rs 1 crore portfolio
   (one row per scenario: % change, P&L in Rs, and loss / 10-day VaR)

For comparison: 1-day 99% VaR = Rs 190,491, 10-day 99% VaR = Rs 602,385
'Loss / 10-day VaR' above 1 means the crisis loss was bigger than VaR predicted.

Worst 5 days for this portfolio:
  2020-03-23   -10.85%   Rs -1,084,558
  2020-03-12    -6.93%   Rs -692,972
  2020-03-16    -6.85%   Rs -685,480
  2020-03-09    -4.80%   Rs -479,588
  2015-08-24    -4.41%   Rs -441,390

Saved table and chart in 'outputs/'
```

Copy your own scenario table into this page from the top of your terminal output or from `outputs/stress_test.csv`.



Figure 11. Your five worst days against the 1-day and 10-day 99% VaR.

How to read it

- **23 March 2020: −10.85% (Rs 10.85 lakh) = 5.7× the 1-day VaR**, and bigger than the 10-day VaR of Rs 6.02 lakh. One day of COVID lost more than VaR predicted for two weeks.

- **4 of the 5 worst days in 11 years fell in one month** (March 2020). Crises cluster, which is why the backtest exceptions bunch together.
- **24 Aug 2015** was the global “Black Monday” after China devalued the yuan.
- **4 June 2024** (election result day) is not in the top 5; gold and USD likely cushioned the stock fall.

STEP 7

The interactive dashboard

app.py — `python -m streamlit run src/app.py`

Turns the whole project into a web app. Move the sliders and every VaR number and chart updates instantly.

src/app.py

```python
# app.py - STEP 7: Interactive VaR dashboard
# Run from the VAR folder:  streamlit run src/app.py

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import streamlit as st
from scipy.stats import norm

from config import PORTFOLIO, PORTFOLIO_VALUE

st.set_page_config(page_title="VaR Risk Dashboard", layout="wide")
st.title("Portfolio Value at Risk Dashboard")
st.caption("8 NSE large caps + Gold ETF + USD | daily data from 2015")

# 1. Load the asset returns from Step 1
returns = pd.read_csv("data/returns.csv", index_col=0, parse_dates=True)

# 2. Sidebar: the user's choices
st.sidebar.header("Settings")
value_lakh = st.sidebar.number_input("Portfolio value (Rs lakh)", 1, 10000, int(PORTFOLIO_VALUE / 100000))
value = value_lakh * 100000
cl = st.sidebar.selectbox("Confidence level", [0.95, 0.99], index=1, format_func=lambda x: f"{x:.0%}")
days = st.sidebar.selectbox("Horizon (days)", [1, 10])

st.sidebar.subheader("Weights (%)")
raw = {}
for ticker, w in PORTFOLIO.items():
    raw[ticker] = st.sidebar.slider(ticker, 0, 50, int(w * 100))

total = sum(raw.values())
if total == 0:
    st.error("Set at least one weight above 0.")
    st.stop()
weights = pd.Series(raw)[returns.columns] / total          # rescale so they add to 100%
st.sidebar.caption(f"Weights total {total}% - rescaled to 100%.")

# 3. Portfolio returns with the chosen weights
port = (returns * weights).sum(axis=1)
scale = np.sqrt(days)

# 4. VaR and ES with three methods
cutoff = np.percentile(port, (1 - cl) * 100)
hist_var = -cutoff * scale
hist_es = -port[port <= cutoff].mean() * scale

z = norm.ppf(1 - cl)
```

```
para_var = -(port.mean() + z * port.std()) * scale
para_es = -(port.mean() - port.std() * norm.pdf(z) / (1 - cl)) * scale

np.random.seed(42)
sims = np.random.multivariate_normal(returns.mean(), returns.cov(), 10000) @ weights.values
mc_cut = np.percentile(sims, (1 - cl) * 100)
mc_var = -mc_cut * scale
mc_es = -sims[sims <= mc_cut].mean() * scale

# 5. Headline numbers
st.subheader(f"{days}-day {cl:.0%} risk on Rs {value_lakh:,} lakh")
c1, c2, c3 = st.columns(3)
c1.metric(f"Historical VaR ({hist_var:.2%})", f"Rs {hist_var * value / 100000:.2f} lakh")
c2.metric(f"Parametric VaR ({para_var:.2%})", f"Rs {para_var * value / 100000:.2f} lakh")
c3.metric(f"Monte Carlo VaR ({mc_var:.2%})", f"Rs {mc_var * value / 100000:.2f} lakh")

table = pd.DataFrame({
    "Method": ["Historical", "Parametric (Normal)", "Monte Carlo (Normal)"],
    "VaR (Rs)": [hist_var * value, para_var * value, mc_var * value],
    "Expected Shortfall (Rs)": [hist_es * value, para_es * value, mc_es * value],
}).round(0)
st.dataframe(table, hide_index=True, width="stretch")

# 6. Charts
left, right = st.columns(2)

with left:
    st.markdown("**Distribution of daily returns**")
    fig, ax = plt.subplots(figsize=(6, 4))
    ax.hist(port * 100, bins=100, color="lightgray", edgecolor="gray")
    ax.axvline(cutoff * 100, color="red", label=f"Historical 1-day VaR {-cutoff:.2%}")
    ax.set_xlabel("Daily return (%)")
    ax.set_ylabel("Number of days")
    ax.legend()
    st.pyplot(fig)

with right:
    st.markdown("**Share of risk by asset**")
    cov = returns.cov()
    port_std = np.sqrt(weights @ cov @ weights)
    contrib = (weights * (cov @ weights) / port_std**2 * 100).sort_values()
    fig, ax = plt.subplots(figsize=(6, 4))
    ax.barh(contrib.index, contrib.values, color="crimson")
    ax.axvline(0, color="black", linewidth=0.8)
    ax.set_xlabel("% of portfolio risk")
    st.pyplot(fig)

# 7. EWMA VaR over time with exceptions
st.markdown("**1-day EWMA VaR over time (red dots = losses bigger than VaR)**")
ewma_std = np.sqrt((port ** 2).ewm(alpha=0.06).mean()).shift(1)
ewma_var = -(z * ewma_std)
exceptions = port[port < -ewma_var]

fig, ax = plt.subplots(figsize=(12, 3.5))
ax.plot(port.index, port * 100, color="gray", linewidth=0.5, label="Daily return")
ax.plot(ewma_var.index, -ewma_var * 100, color="blue", linewidth=1, label=f"-VaR {cl:.0%}")
ax.scatter(exceptions.index, exceptions * 100, color="red", s=10, label=f"Exceptions
({len(exceptions)})")
ax.set_ylabel("Return (%)")
ax.legend(loc="lower left")
st.pyplot(fig)

st.caption("Built in Python with pandas, NumPy, SciPy and Streamlit. Data: Yahoo Finance.")
```

Code	What it does
`import streamlit as st`	The dashboard library.
`st.set_page_config(layout="wide")`	Use the full browser width.
`st.sidebar.number_input / selectbox / slider`	Controls in the left sidebar: portfolio value, confidence, horizon and one weight slider per asset.
`pd.Series(raw)[returns.columns] / total`	Rescale the slider weights so they always add up to 100%.
`st.stop()`	Stop the page early if every weight is 0.
`st.columns(3) / c1.metric(...)`	Three big number cards side by side.
`st.dataframe(table, hide_index=True, width="stretch")`	An interactive table that fills the page width.
`with left: ... st.pyplot(fig)`	Draw a matplotlib chart inside the left column.
`ewm(alpha=0.06)`	EWMA with λ = 0.94 (alpha = 1 − λ), shown over time with exceptions in red.

How Streamlit works

Every time you move a slider, Streamlit **reruns the whole script from top to bottom** with the new value. That is why the code reads like a normal script with no special event handling.

Running it

TERMINAL

```
PS C:\...\VaR> python -m streamlit run src/app.py
  Welcome to Streamlit!
  ...
  Email:            <- just press Enter

  You can now view your Streamlit app in your browser.
  Local URL: http://localhost:8501
```

Why not just "streamlit run"?

Typing `streamlit run src/app.py` gave *"The term 'streamlit' is not recognized"* because pip installed streamlit.exe in a folder that is not on Windows' PATH. `python -m streamlit` runs it through Python, which Windows can always find. Stop the app with **Ctrl + C** in the terminal.

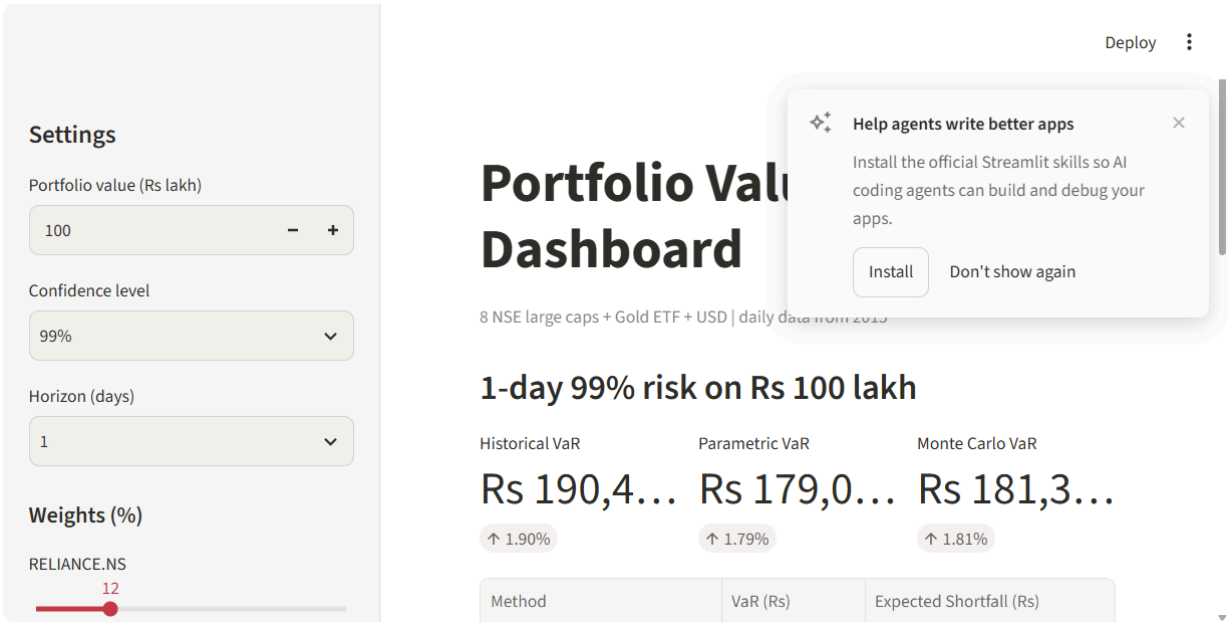


Figure 12. Your first run of the dashboard, before the number cards were changed to show lakh (the cut-off numbers and arrows are now fixed).

Try this	What you will learn
Set ICICI and HDFC to 0	How much of the risk is the banking concentration
Raise Gold to 40%	VaR falls while gold's share of risk stays small
Set Gold and USD to 0	The cost of not diversifying
Switch to 95%	Parametric overtakes historical: the fat-tails crossover

PART 3

Results and reference

What the project proved, what went wrong along the way, and everything you need to look up quickly.

19. The ten findings of this project

1. **Headline risk.** 1-day 99% VaR is **Rs 1.90 lakh** (1.90%) on Rs 1 crore; 10-day VaR is Rs 6.02 lakh.

2. **Fat tails.** Parametric VaR is higher at 95% (1.25% vs 1.10%) but lower at 99% (1.79% vs 1.90%): real returns have fatter tails than a bell curve.

3. **Better tail model.** Student-t Monte Carlo gives 2.03% at 99%, closest to historical; normal models sit at about 1.8%.

4. **Tail depth.** Expected Shortfall at 97.5% exceeds VaR; the Student-t ES is 2.14% (Rs 2.14 lakh).

5. **No model passes.** All three backtested models fail Kupiec (1.55–1.91% exceptions vs 1% target) and sit in the Basel Yellow zone.

6. **Ghost effect.** Historical VaR stayed high for exactly a year after March 2020, then fell overnight when the crash left the window.

7. **Speed vs accuracy.** EWMA reacts fastest to new volatility but breaches most often overall (59 times).

8. **Diversification.** Diversification cuts VaR by **43.8%** (Rs 3.26 lakh → Rs 1.83 lakh). Gold is 12% of capital but 1.9% of risk; USD is a hedge (−0.4%).

9. **Concentration.** Stocks are 78% of capital but about 98% of risk; HDFC + ICICI are 22% of capital but about 30% of risk.

10. **Crisis risk.** The worst day (23 Mar 2020, −10.85%) was 5.7× the 1-day VaR and larger than the 10-day VaR; 4 of the 5 worst days were in March 2020.

20. Errors we hit and how we fixed them

Problem	Cause	Fix
Portfolio “lost” 54.72% in a day; chart axis ±60%	A bad price in the Yahoo data (split or glitch)	Step 3b filter removes any day with a move over 20%
Same output after editing the file	The old version of the file ran: the edit was not saved	Save with Ctrl + S; the tab dot (●) disappears when saved
Red squiggles and “2 problems” after pasting	**IndentationError**: pasted lines started 4 spaces too far right	Replace the whole file (Ctrl + A, Delete, paste). Top-level lines start at the left edge; lines inside `if` are indented 4 spaces
Shift + Tab did nothing	A VS Code extension took over the shortcut	Replace the whole file instead
“streamlit is not recognized”	streamlit.exe is not on Windows PATH	Use `python -m streamlit run src/app.py`
ConnectionResetError (WinError 10054)	A browser tab reconnecting to Streamlit on Windows	Harmless; ignore it
Script seems stuck after a chart appears	`plt.show()` waits for the chart window	Close the chart window
FileNotFoundError: data/...	Step 1 not run, or terminal inside src	Run data_loader.py first, from the VaR folder
Dashboard numbers cut off (“Rs 190,4...”)	Full rupee numbers too wide for the cards	Show values in lakh and move % into the label

21. Command and formula cheat sheet

Commands (run from the VaR folder, in this order)

Command	What it does
`pip install yfinance pandas numpy scipy matplotlib streamlit`	Install everything (once)
`python src/data_loader.py`	Download and clean data → data/
`python src/var_basic.py`	Historical and parametric VaR
`python src/var_advanced.py`	Expected Shortfall and Monte Carlo
`python src/backtest.py`	Backtesting and EWMA
`python src/risk_decomposition.py`	Risk contributions
`python src/stress_test.py`	Stress tests
`python -m streamlit run src/app.py`	Launch the dashboard (Ctrl + C to stop)
`cd ..`	Go up one folder (if the terminal is inside src)

Formulas

Concept	Formula	Python
Log return	$\ln(P_t / P_{t-1})$	`np.log(p / p.shift(1))`
Annual volatility	$\sigma_{daily} \times \sqrt{252}$	`r.std() * np.sqrt(252)`
Portfolio return	$\Sigma\, w_i\, r_i$	`(r * w).sum(axis=1)`
Portfolio volatility	$\sqrt{(w'\Sigma w)}$	`np.sqrt(w @ cov @ w)`
Historical VaR	$-\text{percentile}(r, 1-c)$	`-np.percentile(r, 1)`
Parametric VaR	$-(\mu + z\sigma)$	`-(m + norm.ppf(0.01) * s)`
Expected Shortfall	$-\text{mean}(r \mid r \le -\text{VaR})$	`-r[r <= cut].mean()`
N-day VaR	$\text{VaR}_1 \times \sqrt{N}$	`var * np.sqrt(10)`
EWMA variance	$\sigma^2_t = \lambda\sigma^2_{t-1} + (1-\lambda)r^2_{t-1}$	`(r**2).ewm(alpha=0.06).mean()`
Component VaR	$w_i \times z(\Sigma w)_i / \sigma_p$	`w * z * (cov @ w) / sp`

z-scores

Confidence	90%	95%	97.5%	99%	99.9%
z	−1.282	−1.645	−1.960	−2.326	−3.090

22. Glossary

Backtesting	Comparing past VaR predictions with what actually happened.
Basel / FRTB	Global banking rules. FRTB (Fundamental Review of the Trading Book) replaced 99% VaR with 97.5% ES for trading risk.
Component VaR	An asset's share of total VaR; the components add up to the total.
Correlation	How two assets move together, from −1 to +1.
Covariance matrix (Σ)	A table holding every asset's variance and every pair's covariance.
Diversification benefit	Undiversified VaR minus diversified VaR.
EWMA	Exponentially Weighted Moving Average: volatility where recent days count more (RiskMetrics, $\lambda = 0.94$).
Exception (breach)	A day when the actual loss exceeded VaR.
Expected Shortfall (ES, CVaR)	Average loss on days beyond VaR.
Fat tails	Extreme moves happen more often than a normal distribution predicts.
Ghost effect	A sudden change in historical VaR when an old extreme day leaves the window.
Kupiec test	Statistical test of whether the number of exceptions matches the confidence level.
Kurtosis (excess)	A measure of tail fatness; 0 for a normal distribution.
Log return	ln(today's price / yesterday's price).
Marginal VaR	How much portfolio VaR changes when a little more of one asset is added.
Monte Carlo	Estimating risk by simulating thousands of random scenarios.
Notional	An assumed portfolio value used to express risk in currency.
Skewness	Lopsidedness of a distribution; negative = bigger crashes than rallies.
Stress test	Measuring the loss in a specific severe scenario.
Student-t distribution	A bell-like distribution with fatter tails; fewer degrees of freedom = fatter tails.

Value at Risk (VaR)	The loss that should not be exceeded with a given confidence over a given horizon.
Volatility	Standard deviation of returns.
z-score	Number of standard deviations from the mean.

23. Where to go next

Idea	Why it is worth doing
Filtered Historical Simulation (or EWMA with Student-t)	A model that can pass the backtest: combines EWMA's speed with historical fat tails. Showing the problem and then the fix makes the project stand out.
Christoffersen test	Formally tests whether exceptions cluster.
Add a bond ETF	See how a third asset class changes diversification.
GARCH volatility (the `arch` library)	A more advanced model of changing volatility.
README and GitHub	Upload the project with a README (what it does, how to run it, key findings and charts) and a requirements.txt.
Project report	Turn section 19 into a 5–6 page write-up for interviews and LinkedIn.

How to talk about this project in an interview

"I built a VaR engine for a Rs 1 crore multi-asset Indian portfolio in Python. I compared historical, parametric, EWMA and Monte Carlo VaR, added Expected Shortfall, and backtested with the Kupiec test and the Basel traffic light. No model passed, which I traced to fat tails and the ghost effect after COVID. Risk decomposition showed that stocks were 78% of capital but 98% of risk, and that diversification cut VaR by 44%. Stress tests showed the worst COVID day lost 5.7 times the VaR. I wrapped it all in a Streamlit dashboard."